

Trie (Prefix Tree)

1. Overview

A **Trie** (pronounced "try"), also called a **Prefix Tree**, is a tree-based data structure used to store a dynamic set of strings, where keys are usually strings. Unlike a binary search tree where each node holds a complete key, a Trie stores characters along the edges from root to leaf, and each node represents a **prefix** of one or more keys.

Tries are the foundation of string algorithms in competitive programming and are used extensively in:

- **Autocomplete** systems
- **Spell checkers**
- **IP routing** (longest prefix matching)
- **Dictionary implementations**
- **Pattern matching** (Aho-Corasick automaton)
- **Bit manipulation** problems (XOR Tries)

The name "Trie" comes from the word "retrieval" — a Trie is designed for fast **retrieval** of strings.

2. Intuition

Simple Explanation

A Trie is like a phone book where instead of storing all names in a flat list, you organize them by their letters. All names starting with "A" go under one branch, all names starting with "B" under another, and so on. Within "A", names starting with "Al" form a sub-branch, and so on.

Analogy: A Decision Tree for Letters

Imagine you're playing a game of "20 questions" where you guess a word one letter at a time. You start with nothing. You guess the first letter — if the word starts with 'c', you go down the 'c' path. Then you guess the second letter — 'a' sends you to 'ca', 'o' to 'co', and so on. Eventually, after guessing enough letters, you've uniquely identified the word.

A Trie is exactly this: a tree where each node branches out by the next character. A path from root to a leaf (or a specially marked node) spells out a stored word.

Step-by-Step Reasoning

1. Start with an empty root node (no character).
2. For each word to insert, walk through it character by character.
3. For each character, check if a child node exists for that character.
4. If it exists, move to it. If not, create a new node and move to it.
5. After processing all characters, mark the final node as "end of word".
6. To search, follow the same path. If at any point a child doesn't exist, the word is not in the Trie.
7. If you reach the end and the node is marked "end of word", the word exists.

Why It Works

A Trie exploits the overlapping structure of prefixes. If two words share a prefix (e.g., "cat" and "catastrophe" share "cat"), they share the same nodes in the Trie. This means:

- **Prefix queries** are $O(L)$ — just follow the characters.
- **Insertion** is $O(L)$ — no rebalancing, no resizing, no hashing.
- **Space** is shared among common prefixes, which can be more efficient than storing all strings independently.

3. When to Use It

Use a Trie when:

- **Prefix matching** is the core operation (autocomplete, search suggestions).
- **Multiple strings share common prefixes** and you want space efficiency.
- **You need to check existence of a word or prefix in $O(L)$ time** regardless of how many strings are stored.
- **You need to process strings character by character** (lexicographic ordering, traversal).
- **You need to find all strings with a given prefix** (dictionary search).
- **You need to solve bit-manipulation problems** involving XOR of elements (XOR Trie).

Common Trigger Phrases in Problems

- "Prefix of the string"
- "Autocomplete / auto-suggest"
- "Search suggestions"
- "Longest common prefix"
- "Word dictionary"
- "Word break / segment the string"
- "Maximum XOR of two elements"
- "All words with a given prefix"
- "Pattern matching with multiple patterns"
- "Replace words / filter words by prefix"

4. When Not to Use It

Situation	Why Trie is not suitable	Alternative
Small number of strings	Overhead of 26(256) pointers per node is wasteful	HashSet / sorting
Single string search	No benefit over hashing	Hash map
Exact match only, no prefix queries	Over-engineered; hash set is simpler and faster	HashSet
Memory is extremely tight	Each node has 26+ pointers; can be heavy	Radix tree (PATRICIA), sorted array + binary search

Situation	Why Trie is not suitable	Alternative
Strings are very long but few	$O(L)$ per operation is fine, but space overhead dominates	Hash map
Need to order strings by frequency	Trie doesn't store frequency efficiently	Heap + hash map
Need substring search (not prefix)	Trie only handles prefixes efficiently	Suffix tree / suffix array / KMP
Fuzzy search / edit distance	Trie needs modifications for this	BK-tree, Levenshtein automaton

Common Wrong Assumptions

- "Trie is always faster than hash map" — For exact lookups on small datasets, hash maps are faster due to cache locality.
- "Trie is always memory efficient" — The node overhead (children array) can be huge for sparse alphabets.
- "Trie replaces all string search" — Tries handle prefixes, not arbitrary substrings.

5. Core Concepts

5.1 Trie Node Structure

What it is: Each node in a Trie represents a single character (or prefix) and contains:

- An array/pointer to children (one per possible character in the alphabet).
- A boolean flag `isEnd` marking whether this node represents the end of a complete word.

Why it matters: The node structure is the foundation of everything. The children array enables $O(1)$ lookup for the next character. The `isEnd` flag distinguishes between a prefix and a complete word.

Example:

```
Root node:
  children['a'] -> Node('a')
  children['b'] -> Node('b')
  ... etc.
```

5.2 Trie Insert

What it is: The process of adding a word to the Trie.

Algorithm:

1. Start at root.
2. For each character `c` in the word:

- If `c` is not a child of current node, create a new node.
 - Move to the child node corresponding to `c`.
3. After processing all characters, mark the current node as `isEnd = true`.

Why it matters: Insertion builds the Trie. The same traversal logic is shared with search and prefix queries.

Example:

```
Insert "cat":  
Root -> 'c' -> 'a' -> 't' (mark end)  
Insert "car":  
Root -> 'c' -> 'a' -> 'r' (mark end)  
      \-> 't' is already there
```

5.3 Trie Search

What it is: Checking if a word exists in the Trie.

Algorithm:

1. Start at root.
2. For each character `c` in the word:
 - If `c` is not a child, return false.
 - Move to the child.
3. After processing all characters, return `isEnd` of the current node.

Why it matters: Search is $O(L)$ where L is the word length, independent of the number of words stored. This is the key promise of a Trie.

Edge case: Searching for "ca" when "cat" exists — you reach node 'a' but `isEnd` is false, so it returns false correctly.

5.4 Prefix Search (StartsWith)

What it is: Checking if any word in the Trie starts with a given prefix.

Algorithm:

1. Start at root.
2. For each character `c` in the prefix:
 - If `c` is not a child, return false.
 - Move to the child.
3. If we successfully processed all characters, return true (the prefix exists).

Why it matters: This is what makes Tries unique. Prefix search is $O(L)$ and doesn't need `isEnd` — just existence of the path.

Example: In a Trie containing "catastrophe", searching prefix "cat" returns true, even though "cat" may not be a complete word.

5.5 Word Dictionary (LeetCode 208)

What it is: The classic "Implement Trie" problem — building a Trie class with `insert`, `search`, and `startsWith` methods.

Why it matters: This is the most common interview question on Tries. Most Trie problems are built on top of this foundation.

Key insight: The `search` method checks `isEnd`, while `startsWith` only checks if the path exists.

5.6 Autocomplete (Suggestions)

What it is: Given a prefix, find all words in the Trie that start with that prefix.

Algorithm:

1. Find the node corresponding to the prefix (prefix search).
2. From that node, perform a DFS/BFS to collect all paths that end at a word.
3. Return the collected words (prefix + suffix).

Why it matters: This is the core feature of search engines, text editors, and keyboards. The Trie structure makes this efficient because all words with the given prefix are in the subtree of the prefix node.

Example: Prefix "ca" in a Trie with {"cat", "car", "cab", "dog"} → {"cat", "car", "cab"}.

5.7 Delete from Trie

What it is: Removing a word from the Trie while cleaning up unused nodes.

Algorithm:

1. Recursively traverse to the end of the word.
2. Unmark `isEnd`.
3. On the way back up, if a node has no children and is not the end of another word, delete it.

Why it matters: Deletion must be careful not to remove nodes that are used by other words sharing the same prefix.

Example: Deleting "cat" from a Trie with {"cat", "catastrophe"} — the 'c', 'a', 't' nodes are shared, so only the `isEnd` flag on 't' is removed. The nodes remain because "catastrophe" needs them.

5.8 XOR Trie (Binary Trie)

What it is: A Trie that stores numbers in their binary representation (as 32-bit or 64-bit strings). Each node has at most 2 children: 0 and 1.

Why it matters: This enables solving maximum XOR pair problems in $O(N * B)$ where B is the number of bits. It exploits the Trie's prefix property on bit prefixes.

Key insight: To maximize XOR, at each bit position we want the opposite bit. If the current bit is 0, we want 1; if it's 1, we want 0. The XOR Trie lets us greedily find the best opposite bit in $O(1)$ per bit.

5.9 Maximum XOR Pair (LeetCode 421)

What it is: Given an array of numbers, find the maximum XOR of any two numbers.

Algorithm:

1. Insert all numbers into a binary Trie (32-bit representation).
2. For each number, query the Trie for the maximum XOR possible:
 - At each bit, try to go to the opposite child ($0 \rightarrow 1$, $1 \rightarrow 0$).
 - If the opposite child exists, take it (this bit contributes to XOR).
 - If not, take the same bit.
3. Track the maximum.

Why it matters: This is $O(N * 32)$ instead of $O(N^2)$ naive. A classic problem showing the power of binary Tries.

5.10 Word Break Using Trie

What it is: Given a string and a dictionary of words, determine if the string can be segmented into dictionary words.

Trie-based approach:

1. Insert all dictionary words into a Trie.
2. Use DP with Trie: $dp[i] = \text{true}$ if $s[0..i-1]$ can be segmented.
3. For each position i where $dp[i]$ is true, traverse the Trie from $s[i]$ onwards, marking $dp[j] = \text{true}$ for each position j where we hit a word end.

Why it matters: The Trie approach is faster than the naive DP + HashSet approach (which checks all substrings) because it prunes invalid prefixes early.

Comparison: HashSet DP is $O(N^2)$ worst case; Trie DP is $O(N^2)$ worst case too but with better constants and early termination.

5.11 Persistent Trie

What it is: A versioned Trie where each insertion creates a new root, but unchanged subtrees are shared between versions. This allows querying any previous version of the Trie.

Why it matters: Persistent data structures are crucial for problems involving time travel, range queries, or queries on historical data. A persistent Trie can answer "what was the Trie state at version V ?"

Key insight: When inserting a word, only the nodes along the path are newly created. All other child pointers point to nodes from the previous version. This makes each version $O(L)$ space instead of $O(\text{total nodes})$.

Applications:

- Range queries on XOR (find max XOR in subarray)
- Versioned dictionaries
- Rollback in string algorithms

5.12 Aho-Corasick Automaton

What it is: A **multi-pattern string matching algorithm** that builds a Trie from all patterns, then adds **failure links** (suffix links) to create a finite automaton. It can find all occurrences of all patterns in a text in $O(N + M + \text{occurrences})$.

K) time where N is text length, M is total pattern length, and K is number of matches.

Why it matters: Aho-Corasick is the ultimate extension of Tries. It's the go-to for problems involving multiple pattern matching in a single pass.

Key components:

- **Trie:** Stores all patterns.
- **Failure links:** Each node gets a link to the longest proper suffix that is also a prefix in the Trie. If a match fails, the automaton follows the failure link instead of restarting.
- **Output links:** Optional links to the nearest node that represents a complete pattern (for efficiency).

Algorithm:

1. Build Trie from patterns.
 2. BFS to compute failure links (like KMP's failure function, but for a Trie).
 3. Traverse the text through the automaton, following failure links on mismatch.
 4. At each node, report all patterns that end at this position.
-

6. Step-by-Step Algorithm

6.1 Basic Trie Operations

Insert(word):

1. Set `curr = root`.
2. For each `char c` in `word`:
 - Let `idx = c - 'a'` (0-25 for lowercase).
 - If `curr->children[idx] == nullptr`, create a new node.
 - Set `curr = curr->children[idx]`.
3. Set `curr->isEnd = true`.

Search(word):

1. Set `curr = root`.
2. For each `char c` in `word`:
 - Let `idx = c - 'a'`.
 - If `curr->children[idx] == nullptr`, return `false`.
 - Set `curr = curr->children[idx]`.
3. Return `curr->isEnd`.

StartsWith(prefix):

1. Set `curr = root`.
2. For each `char c` in `prefix`:
 - Let `idx = c - 'a'`.
 - If `curr->children[idx] == nullptr`, return `false`.
 - Set `curr = curr->children[idx]`.
3. Return `true`.

6.2 XOR Trie (Binary Trie)

Insert(num):

1. For bit from 31 down to 0:
 - Let `b = (num >> bit) & 1`.
 - If `curr->child[b] == nullptr`, create a new node.
 - Move to `curr->child[b]`.

Query max XOR:

1. For bit from 31 down to 0:
 - Let `b = (num >> bit) & 1`.
 - If `curr->child[!b]` exists, take it (contributes `1 << bit` to XOR result).
 - Else take `curr->child[b]`.

6.3 Aho-Corasick Construction

1. Build Trie from all patterns.
2. BFS from root:
 - For each node, set failure link = `go(fail[parent], char)`.
 - If failure link is root and the child doesn't exist, set it to root.
 - Set output link: if failure node is a word end, link to it.
3. Search: traverse text, follow failure links on mismatch, report all matches.

7. Dry Run

7.1 Basic Trie Insert and Search

Insert words: "cat", "car", "cab", "dog"

Initial state: Root node with all 26 children = nullptr.

Step 1: Insert "cat"

```
Root -> 'c' (new) -> 'a' (new) -> 't' (new, isEnd=true)
```

Step 2: Insert "car"

```
Root -> 'c' -> 'a' (already exists)
      -> 't' (isEnd=true)
      -> 'r' (new, isEnd=true)
```

Step 3: Insert "cab"

```

Root -> 'c' -> 'a'
      -> 't' (isEnd=true)
      -> 'r' (isEnd=true)
      -> 'b' (new, isEnd=true)

```

Step 4: Insert "dog"

```

Root -> 'c' -> 'a' -> ...
      -> 'd' (new) -> 'o' (new) -> 'g' (new, isEnd=true)

```

Final Trie:

```

      root
     /  \
    c    d
    |    |
    a    o
   / | \  |
  t  r b  g (end)
(end)(end)(end)

```

Search "car": root → 'c' → 'a' → 'r' (isEnd=true) → **Found**
Search "ca": root → 'c' → 'a' (isEnd=false) → **Not found**
Search "can": root → 'c' → 'a' → 'n' doesn't exist → **Not found**

7.2 XOR Trie Maximum XOR

Array: [3, 10, 5, 25, 2, 8]

Step-by-step for 32-bit numbers (showing 5 bits for simplicity):

Binary representation (5 bits):

- 3: 00011
- 10: 01010
- 5: 00101
- 25: 11001
- 2: 00010
- 8: 01000

Insert 3 (00011):

```

root -> bit4:0 -> bit3:0 -> bit2:0 -> bit1:1 -> bit0:1 (end)

```

Insert 10 (01010):

```

root -> 0 -> 0 -> 0 -> 1 -> 1 (3)
      \-> 1 -> 0 -> 1 -> 0 (10)

```

Continue inserting all...

Query max XOR for 25 (11001):

- bit4: 1 → want 0 → exists (3's path) → take 0 → XOR bit = 1
- bit3: 1 → want 0 → exists → take 0 → XOR bit = 1
- bit2: 0 → want 1 → exists → take 1 → XOR bit = 1
- bit1: 0 → want 1 → exists → take 1 → XOR bit = 1
- bit0: 1 → want 0 → exists → take 0 → XOR bit = 1

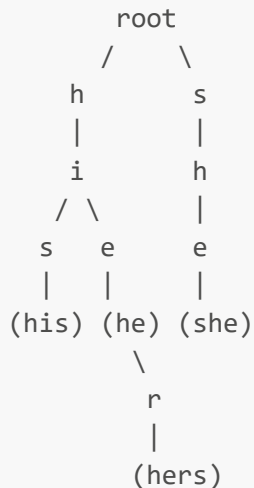
Result: 11111 = 31. 25 XOR 10 = 31. ✓

Maximum XOR pair: 25 XOR 10 = 31

7.3 Aho-Corasick

Patterns: {"he", "she", "his", "hers"} **Text:** "ahishers"

Step 1: Build Trie



Step 2: Failure Links (BFS)

Node	Failure Link
root	root (self)
h	root
s	root
i	root
h (under s)	h (under root)
e (under h)	root
e (under s-h)	e (under h)

```
s (under h-i) | s (under root)
r (under h-e) | root
```

Step 3: Search "ahishers"

Pos	Char	Node	Matches
0	a	root	-
1	h	h	-
2	i	h-i	-
3	s	h-i-s	"his" found at pos 1-3
4	h	h	-
5	e	h-e	"he" found at pos 4-5
6	r	h-e-r	"hers" found at pos 4-6
7	s	s	-

Matches:

- "his" at index 1
- "he" at index 4
- "hers" at index 4

8. C++ Implementation

8.1 Basic Trie (Lowercase English Letters)

```
#include <bits/stdc++.h>
using namespace std;

class TrieNode {
public:
    TrieNode* children[26];
    bool isEnd;

    TrieNode() {
        for (int i = 0; i < 26; i++) children[i] = nullptr;
        isEnd = false;
    }

    ~TrieNode() {
        for (int i = 0; i < 26; i++) {
            if (children[i]) delete children[i];
        }
    }
};
```

```
class Trie {
private:
    TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }

    ~Trie() {
        delete root;
    }

    // Insert a word into the Trie
    void insert(string word) {
        TrieNode* curr = root;
        for (char c : word) {
            int idx = c - 'a';
            if (curr->children[idx] == nullptr) {
                curr->children[idx] = new TrieNode();
            }
            curr = curr->children[idx];
        }
        curr->isEnd = true;
    }

    // Search for a word (exact match)
    bool search(string word) {
        TrieNode* curr = root;
        for (char c : word) {
            int idx = c - 'a';
            if (curr->children[idx] == nullptr) return false;
            curr = curr->children[idx];
        }
        return curr->isEnd;
    }

    // Check if any word starts with the given prefix
    bool startsWith(string prefix) {
        TrieNode* curr = root;
        for (char c : prefix) {
            int idx = c - 'a';
            if (curr->children[idx] == nullptr) return false;
            curr = curr->children[idx];
        }
        return true;
    }

    // Delete a word from the Trie
    bool remove(string word) {
        return removeHelper(root, word, 0);
    }
}
```

```

// Get all words with given prefix (autocomplete)
vector<string> autocomplete(string prefix) {
    TrieNode* curr = root;
    vector<string> result;

    // Navigate to prefix node
    for (char c : prefix) {
        int idx = c - 'a';
        if (curr->children[idx] == nullptr) return result; // no suggestions
        curr = curr->children[idx];
    }

    // DFS to collect all words from this node
    string current = prefix;
    collectWords(curr, current, result);
    return result;
}

// Check if a word can be deleted from the Trie
// Returns true if the word was found and deleted
bool removeHelper(TrieNode* node, string& word, int depth) {
    if (depth == word.size()) {
        if (!node->isEnd) return false;
        node->isEnd = false;
        return isEmpty(node);
    }

    int idx = word[depth] - 'a';
    if (node->children[idx] == nullptr) return false;

    bool shouldDeleteChild = removeHelper(node->children[idx], word, depth +
1);

    if (shouldDeleteChild) {
        delete node->children[idx];
        node->children[idx] = nullptr;
        return !node->isEnd && isEmpty(node);
    }

    return false;
}

bool isEmpty(TrieNode* node) {
    for (int i = 0; i < 26; i++) {
        if (node->children[i] != nullptr) return false;
    }
    return true;
}

void collectWords(TrieNode* node, string& current, vector<string>& result) {
    if (node->isEnd) result.push_back(current);
    for (int i = 0; i < 26; i++) {
        if (node->children[i] != nullptr) {
            current.push_back('a' + i);

```

```

        collectWords(node->children[i], current, result);
        current.pop_back();
    }
}
};

// Example usage
int main() {
    Trie trie;

    vector<string> words = {"cat", "car", "cab", "dog", "catastrophe", "catalog"};
    for (string& w : words) trie.insert(w);

    cout << "Search 'cat': " << trie.search("cat") << endl;           // 1
    cout << "Search 'ca': " << trie.search("ca") << endl;             // 0
    cout << "StartsWith 'ca': " << trie.startsWith("ca") << endl;     // 1

    cout << "Autocomplete 'ca': ";
    vector<string> suggestions = trie.autocomplete("ca");
    for (string& s : suggestions) cout << s << " ";
    cout << endl;
    // cat car cab catastrophe catalog

    trie.remove("cat");
    cout << "Search 'cat' after delete: " << trie.search("cat") << endl; // 0
    cout << "Autocomplete 'cat': ";
    suggestions = trie.autocomplete("cat");
    for (string& s : suggestions) cout << s << " ";
    cout << endl;
    // catastrophe catalog (cat removed, but "catastrophe" and "catalog" share
    // prefix "cat")

    return 0;
}

```

8.2 XOR Trie (Binary Trie)

```

#include <bits/stdc++.h>
using namespace std;

struct BinaryTrieNode {
    BinaryTrieNode* child[2];
    int count; // number of numbers passing through this node

    BinaryTrieNode() {
        child[0] = child[1] = nullptr;
        count = 0;
    }
};

```

```

class BinaryTrie {
private:
    BinaryTrieNode* root;
    static const int BITS = 31; // for 32-bit integers (0 to 30 for non-negative)

public:
    BinaryTrie() {
        root = new BinaryTrieNode();
    }

    ~BinaryTrie() {
        clear(root);
    }

    void clear(BinaryTrieNode* node) {
        if (node->child[0]) clear(node->child[0]);
        if (node->child[1]) clear(node->child[1]);
        delete node;
    }

    // Insert a number into the binary trie
    void insert(int num) {
        BinaryTrieNode* curr = root;
        for (int bit = BITS; bit >= 0; bit--) {
            int b = (num >> bit) & 1;
            if (curr->child[b] == nullptr) {
                curr->child[b] = new BinaryTrieNode();
            }
            curr = curr->child[b];
            curr->count++;
        }
    }

    // Remove a number from the binary trie
    void remove(int num) {
        BinaryTrieNode* curr = root;
        for (int bit = BITS; bit >= 0; bit--) {
            int b = (num >> bit) & 1;
            if (curr->child[b] == nullptr) return;
            curr = curr->child[b];
            curr->count--;
        }
    }

    // Query maximum XOR possible with given number
    int queryMaxXor(int num) {
        BinaryTrieNode* curr = root;
        int result = 0;
        for (int bit = BITS; bit >= 0; bit--) {
            int b = (num >> bit) & 1;
            int desired = 1 - b; // we want opposite bit for maximum XOR
            if (curr->child[desired] != nullptr && curr->child[desired]->count >
0) {
                result |= (1 << bit);
            }
        }
    }
}

```

```

        curr = curr->child[desired];
    } else {
        curr = curr->child[b];
    }
}
return result;
}

// Find the maximum XOR pair in the array
int findMaxXorPair(vector<int>& arr) {
    int maxXor = 0;
    for (int num : arr) {
        insert(num);
        maxXor = max(maxXor, queryMaxXor(num));
    }
    return maxXor;
}

};

// Example usage
int main() {
    vector<int> arr = {3, 10, 5, 25, 2, 8};
    BinaryTrie bt;

    cout << "Maximum XOR pair: " << bt.findMaxXorPair(arr) << endl; // 31

    // Alternative: find max XOR of any two numbers
    BinaryTrie bt2;
    int maxXor = 0;
    for (int num : arr) {
        bt2.insert(num);
    }
    for (int num : arr) {
        maxXor = max(maxXor, bt2.queryMaxXor(num));
    }
    cout << "Maximum XOR: " << maxXor << endl; // 31

    return 0;
}

```

8.3 Word Break Using Trie

```

#include <bits/stdc++.h>
using namespace std;

class TrieNode {
public:
    TrieNode* children[26];
    bool isEnd;

    TrieNode() {

```

```

        for (int i = 0; i < 26; i++) children[i] = nullptr;
        isEnd = false;
    }
};

class Trie {
public:
    TrieNode* root;

    Trie() { root = new TrieNode(); }

    void insert(string word) {
        TrieNode* curr = root;
        for (char c : word) {
            int idx = c - 'a';
            if (curr->children[idx] == nullptr)
                curr->children[idx] = new TrieNode();
            curr = curr->children[idx];
        }
        curr->isEnd = true;
    }
};

// Word Break using Trie + DP
bool wordBreak(string s, vector<string>& wordDict) {
    Trie trie;
    for (string& word : wordDict) trie.insert(word);

    int n = s.size();
    vector<bool> dp(n + 1, false);
    dp[0] = true; // empty string is always breakable

    for (int i = 0; i < n; i++) {
        if (!dp[i]) continue; // s[0..i-1] cannot be segmented

        TrieNode* curr = trie.root;
        for (int j = i; j < n; j++) {
            int idx = s[j] - 'a';
            if (curr->children[idx] == nullptr) break; // no prefix matches
            curr = curr->children[idx];
            if (curr->isEnd) {
                dp[j + 1] = true; // s[i..j] is a word
            }
        }
    }

    return dp[n];
}

// Example usage
int main() {
    string s = "leetcode";
    vector<string> wordDict = {"leet", "code"};
    cout << "Word break: " << wordBreak(s, wordDict) << endl; // 1 (true)
}

```

```

    s = "applepenapple";
    wordDict = {"apple", "pen"};
    cout << "Word break: " << wordBreak(s, wordDict) << endl; // 1 (true)

    s = "catsandog";
    wordDict = {"cats", "dog", "sand", "and", "cat"};
    cout << "Word break: " << wordBreak(s, wordDict) << endl; // 0 (false)

    return 0;
}

```

8.4 Aho-Corasick Automaton

```

#include <bits/stdc++.h>
using namespace std;

struct AhoCorasickNode {
    // child[26] for lowercase letters
    AhoCorasickNode* child[26];
    AhoCorasickNode* fail;    // failure link
    vector<int> output;        // indices of patterns ending at this node
    int depth;                // depth of this node (for debugging)

    AhoCorasickNode() {
        for (int i = 0; i < 26; i++) child[i] = nullptr;
        fail = nullptr;
        depth = 0;
    }
};

class AhoCorasick {
private:
    AhoCorasickNode* root;
    vector<string> patterns;

public:
    AhoCorasick() {
        root = new AhoCorasickNode();
    }

    ~AhoCorasick() {
        clear(root);
    }

    void clear(AhoCorasickNode* node) {
        for (int i = 0; i < 26; i++) {
            if (node->child[i]) clear(node->child[i]);
        }
        delete node;
    }
}

```

```

// Insert a pattern into the Trie
void insert(string pattern, int index) {
    AhoCorasickNode* curr = root;
    for (char c : pattern) {
        int idx = c - 'a';
        if (curr->child[idx] == nullptr) {
            curr->child[idx] = new AhoCorasickNode();
            curr->child[idx]->depth = curr->depth + 1;
        }
        curr = curr->child[idx];
    }
    curr->output.push_back(index);
}

// Build failure links using BFS
void build() {
    queue<AhoCorasickNode*> q;

    // Initialize failure links for depth-1 nodes (first level)
    for (int i = 0; i < 26; i++) {
        if (root->child[i]) {
            root->child[i]->fail = root;
            q.push(root->child[i]);
        } else {
            root->child[i] = root; // optimization: missing edges point to
root
        }
    }

    while (!q.empty()) {
        AhoCorasickNode* curr = q.front();
        q.pop();

        for (int i = 0; i < 26; i++) {
            AhoCorasickNode* next = curr->child[i];
            if (next && next != root) {
                // Set failure link: go to fail[curr] and find child with same
char
                AhoCorasickNode* f = curr->fail;
                while (f != root && f->child[i] == root) {
                    f = f->fail;
                }
                next->fail = (f->child[i] && f->child[i] != root) ? f->
>child[i] : root;

                // Merge output: if failure node is a word end, add its
patterns
                next->output.insert(next->output.end(),
                    next->fail->output.begin(), next->fail->output.end());

                q.push(next);
            } else if (next == nullptr) {
                // Missing edge optimization: set to failure link's child

```

```

        curr->child[i] = (curr == root) ? root : curr->fail->child[i];
    }
}
}

// Search for all pattern occurrences in text
// Returns vector of (pattern_index, position)
vector<pair<int, int>> search(string text) {
    vector<pair<int, int>> matches;
    AhoCorasickNode* curr = root;

    for (int pos = 0; pos < (int)text.size(); pos++) {
        int idx = text[pos] - 'a';

        // Follow the edge (already optimized with missing edge handling)
        curr = curr->child[idx];

        // Report all patterns ending at this position
        for (int patternIdx : curr->output) {
            matches.push_back({patternIdx, pos -
(int)patterns[patternIdx].size() + 1});
        }
    }

    return matches;
}

void setPatterns(vector<string>& pats) {
    patterns = pats;
    for (int i = 0; i < (int)patterns.size(); i++) {
        insert(patterns[i], i);
    }
    build();
}

};

// Example usage
int main() {
    vector<string> patterns = {"he", "she", "his", "hers"};
    string text = "ahishers";

    AhoCorasick ac;
    ac.setPatterns(patterns);

    auto matches = ac.search(text);

    cout << "Patterns found:" << endl;
    for (auto& [patternIdx, pos] : matches) {
        cout << "  " << patterns[patternIdx] << " at position " << pos << endl;
    }
    // Output:
    //   'his' at position 1
    //   'he' at position 4

```

```

    // 'hers' at position 4

    return 0;
}

```

8.5 Persistent Trie

```

#include <bits/stdc++.h>
using namespace std;

struct PersistentTrieNode {
    PersistentTrieNode* child[26];
    int count; // number of words passing through this node
    bool isEnd;

    PersistentTrieNode() {
        for (int i = 0; i < 26; i++) child[i] = nullptr;
        count = 0;
        isEnd = false;
    }
};

class PersistentTrie {
private:
    vector<PersistentTrieNode*> roots; // roots[0] = empty, roots[i] = after i
    insertions

public:
    PersistentTrie() {
        roots.push_back(new PersistentTrieNode());
    }

    // Insert a word, creating a new version
    // Returns the new root
    PersistentTrieNode* insert(PersistentTrieNode* prevRoot, string word) {
        PersistentTrieNode* newRoot = new PersistentTrieNode();
        PersistentTrieNode* curr = newRoot;
        PersistentTrieNode* prev = prevRoot;

        // Copy root's children from previous version
        for (int i = 0; i < 26; i++) {
            curr->child[i] = prev->child[i];
        }
        curr->count = prev->count + 1;

        for (char c : word) {
            int idx = c - 'a';

            // Create new node for this character
            PersistentTrieNode* newNode = new PersistentTrieNode();
            if (prev->child[idx]) {

```

```

        // Copy the previous node's children
        for (int i = 0; i < 26; i++) {
            newNode->child[i] = prev->child[idx]->child[i];
        }
        newNode->count = prev->child[idx]->count + 1;
        newNode->isEnd = prev->child[idx]->isEnd;
    } else {
        newNode->count = 1;
    }

    curr->child[idx] = newNode;
    curr = curr->child[idx];
    prev = prev->child[idx] ? prev->child[idx] : new PersistentTrieNode();
}

curr->isEnd = true;
return newRoot;
}

// Insert word as a new version
void insert(string word) {
    PersistentTrieNode* newRoot = insert(roots.back(), word);
    roots.push_back(newRoot);
}

// Search in a specific version
bool search(int version, string word) {
    if (version < 0 || version >= (int)roots.size()) return false;
    PersistentTrieNode* curr = roots[version];
    for (char c : word) {
        int idx = c - 'a';
        if (curr->child[idx] == nullptr) return false;
        curr = curr->child[idx];
    }
    return curr->isEnd;
}

// Get number of words with given prefix in a specific version
int countPrefix(int version, string prefix) {
    if (version < 0 || version >= (int)roots.size()) return 0;
    PersistentTrieNode* curr = roots[version];
    for (char c : prefix) {
        int idx = c - 'a';
        if (curr->child[idx] == nullptr) return 0;
        curr = curr->child[idx];
    }
    return curr->count;
}

// Get number of versions
int getVersionCount() { return roots.size(); }
};

// Example usage

```

```

int main() {
    PersistentTrie pt;

    pt.insert("cat");
    pt.insert("car");
    pt.insert("dog");

    // Version 0: empty
    // Version 1: after "cat"
    // Version 2: after "car"
    // Version 3: after "dog"

    cout << "Version 0 search 'cat': " << pt.search(0, "cat") << endl; // 0
    cout << "Version 1 search 'cat': " << pt.search(1, "cat") << endl; // 1
    cout << "Version 2 search 'cat': " << pt.search(2, "cat") << endl; // 1
    cout << "Version 2 search 'car': " << pt.search(2, "car") << endl; // 1
    cout << "Version 3 search 'dog': " << pt.search(3, "dog") << endl; // 1

    cout << "Version 3 count prefix 'ca': " << pt.countPrefix(3, "ca") << endl; //
2 (cat, car)

    return 0;
}

```

9. Python Implementation

9.1 Basic Trie

```

class TrieNode:
    def __init__(self):
        self.children = {} # dictionary for flexibility (supports any alphabet)
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                curr.children[ch] = TrieNode()
            curr = curr.children[ch]
        curr.is_end = True

    def search(self, word: str) -> bool:
        curr = self.root
        for ch in word:
            if ch not in curr.children:

```

```

        return False
    curr = curr.children[ch]
    return curr.is_end

def starts_with(self, prefix: str) -> bool:
    curr = self.root
    for ch in prefix:
        if ch not in curr.children:
            return False
        curr = curr.children[ch]
    return True

def autocomplete(self, prefix: str) -> list:
    """Return all words with given prefix."""
    curr = self.root
    for ch in prefix:
        if ch not in curr.children:
            return []
        curr = curr.children[ch]

    result = []
    self._collect_words(curr, prefix, result)
    return result

def _collect_words(self, node: TrieNode, prefix: str, result: list) -> None:
    if node.is_end:
        result.append(prefix)
    for ch, child in node.children.items():
        self._collect_words(child, prefix + ch, result)

def remove(self, word: str) -> bool:
    """Remove a word from the Trie. Returns True if deleted."""
    def _remove(node: TrieNode, word: str, depth: int) -> bool:
        if depth == len(word):
            if not node.is_end:
                return False
            node.is_end = False
            return len(node.children) == 0 # can delete this node?

        ch = word[depth]
        if ch not in node.children:
            return False

        should_delete = _remove(node.children[ch], word, depth + 1)
        if should_delete:
            del node.children[ch]
            return len(node.children) == 0 and not node.is_end

    return False

    return _remove(self.root, word, 0)

```

Example usage

```

if __name__ == "__main__":
    trie = Trie()
    words = ["cat", "car", "cab", "dog", "catastrophe", "catalog"]
    for w in words:
        trie.insert(w)

    print("Search 'cat':", trie.search("cat"))      # True
    print("Search 'ca':", trie.search("ca"))        # False
    print("StartsWith 'ca':", trie.starts_with("ca")) # True

    print("Autocomplete 'ca':", trie.autocomplete("ca"))
    # ['cat', 'car', 'cab', 'catastrophe', 'catalog']

    trie.remove("cat")
    print("After delete 'cat':", trie.search("cat")) # False
    print("Autocomplete 'cat':", trie.autocomplete("cat"))
    # ['catastrophe', 'catalog']

```

9.2 XOR Trie (Binary Trie)

```

class BinaryTrieNode:
    def __init__(self):
        self.child = [None, None]
        self.count = 0

class BinaryTrie:
    def __init__(self, bits=31):
        self.root = BinaryTrieNode()
        self.BITS = bits # number of bits (31 for 32-bit ints)

    def insert(self, num: int) -> None:
        curr = self.root
        for bit in range(self.BITS, -1, -1):
            b = (num >> bit) & 1
            if curr.child[b] is None:
                curr.child[b] = BinaryTrieNode()
            curr = curr.child[b]
            curr.count += 1

    def remove(self, num: int) -> None:
        curr = self.root
        for bit in range(self.BITS, -1, -1):
            b = (num >> bit) & 1
            if curr.child[b] is None:
                return
            curr = curr.child[b]
            curr.count -= 1

    def query_max_xor(self, num: int) -> int:
        curr = self.root

```

```

        result = 0
        for bit in range(self.BITS, -1, -1):
            b = (num >> bit) & 1
            desired = 1 - b
            if curr.child[desired] is not None and curr.child[desired].count > 0:
                result |= (1 << bit)
                curr = curr.child[desired]
            else:
                curr = curr.child[b]
        return result

    def find_max_xor_pair(self, arr: list) -> int:
        max_xor = 0
        for num in arr:
            self.insert(num)
            max_xor = max(max_xor, self.query_max_xor(num))
        return max_xor

# Example usage
if __name__ == "__main__":
    arr = [3, 10, 5, 25, 2, 8]
    bt = BinaryTrie()
    print("Maximum XOR pair:", bt.find_max_xor_pair(arr)) # 31

```

9.3 Word Break Using Trie

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                curr.children[ch] = TrieNode()
            curr = curr.children[ch]
        curr.is_end = True

    def word_break(s: str, word_dict: list) -> bool:
        trie = Trie()
        for word in word_dict:
            trie.insert(word)

```

```

n = len(s)
dp = [False] * (n + 1)
dp[0] = True # empty string is breakable

for i in range(n):
    if not dp[i]:
        continue

    curr = trie.root
    for j in range(i, n):
        ch = s[j]
        if ch not in curr.children:
            break
        curr = curr.children[ch]
        if curr.is_end:
            dp[j + 1] = True

return dp[n]

# Example usage
if __name__ == "__main__":
    print(word_break("leetcode", ["leet", "code"])) # True
    print(word_break("applepenapple", ["apple", "pen"])) # True
    print(word_break("catsandog", ["cats", "dog", "sand", "and", "cat"])) # False

```

9.4 Aho-Corasick

```

from collections import deque

class AhoCorasickNode:
    def __init__(self):
        self.child = {} # char -> node
        self.fail = None # failure link
        self.output = [] # pattern indices ending here
        self.depth = 0

class AhoCorasick:
    def __init__(self):
        self.root = AhoCorasickNode()
        self.patterns = []

    def add_pattern(self, pattern: str, index: int) -> None:
        curr = self.root
        for ch in pattern:
            if ch not in curr.child:
                curr.child[ch] = AhoCorasickNode()
                curr.child[ch].depth = curr.depth + 1
            curr = curr.child[ch]

```

```

curr.output.append(index)

def build(self) -> None:
    """Build failure links using BFS."""
    queue = deque()

    # Initialize first level
    for ch, node in self.root.child.items():
        node.fail = self.root
        queue.append(node)

    # Also set missing edges from root to root
    # (handled via defaultdict-style logic)

    while queue:
        curr = queue.popleft()

        for ch, next_node in curr.child.items():
            # Find failure link for next_node
            f = curr.fail
            while f is not None and ch not in f.child:
                f = f.fail
            next_node.fail = f.child[ch] if f and ch in f.child else self.root

            # Merge output from failure node
            if next_node.fail:
                next_node.output.extend(next_node.fail.output)

            queue.append(next_node)

def set_patterns(self, patterns: list) -> None:
    self.patterns = patterns
    for i, pat in enumerate(patterns):
        self.add_pattern(pat, i)
    self.build()

def search(self, text: str) -> list:
    """Find all pattern occurrences. Returns list of (pattern_index,
    start_position)."""
    matches = []
    curr = self.root

    for pos, ch in enumerate(text):
        # Follow failure links until we find a match or reach root
        while curr is not self.root and ch not in curr.child:
            curr = curr.fail

        if ch in curr.child:
            curr = curr.child[ch]
        else:
            curr = self.root

    # Report all patterns ending here
    for pat_idx in curr.output:

```

```

        start_pos = pos - len(self.patterns[pat_idx]) + 1
        matches.append((pat_idx, start_pos))

    return matches

# Example usage
if __name__ == "__main__":
    patterns = ["he", "she", "his", "hers"]
    text = "ahishers"

    ac = AhoCorasick()
    ac.set_patterns(patterns)

    matches = ac.search(text)
    print("Patterns found:")
    for pat_idx, pos in matches:
        print(f" '{patterns[pat_idx]}' at position {pos}")
# Output:
# 'his' at position 1
# 'he' at position 4
# 'hers' at position 4

```

10. Code Explanation

10.1 Basic Trie

Node Structure:

- `TrieNode* children[26]` — An array of 26 pointers (one per lowercase letter). If `children[i]` is `nullptr`, no word has that character at this position. This enables $O(1)$ lookup for the next character.
- `bool isEnd` — Marks whether a complete word ends at this node. A node can be both a prefix for longer words and the end of a shorter word.

Insert:

- The loop walks through each character of the word. If a child node doesn't exist for the current character, it creates one. After processing all characters, it sets `isEnd = true`.
- **Why this works:** Common prefixes are shared. If "cat" is already inserted, inserting "car" reuses the 'c' and 'a' nodes.

Search:

- Follows the same path as insert. If at any point a child doesn't exist, the word is not in the Trie. After the loop, it returns `isEnd` — because the path might exist (as a prefix of another word) without the exact word being stored.

StartsWith:

- Same as search but returns `true` as long as the path exists. The `isEnd` check is skipped.

Autocomplete:

- First navigates to the node representing the prefix. Then performs a DFS from that node, collecting all words by tracking the path. When a node with `isEnd = true` is reached, the current path (prefix + accumulated characters) is added to the result.

Delete:

- Recursive post-order traversal. First, it finds the end node. If the word exists, it unmarks `isEnd`. Then, on the way back up, it checks if a node is "dead" (no children and not end of another word). If so, it deletes the node and sets the parent's pointer to `nullptr`.

10.2 XOR Trie (Binary Trie)

Node Structure:

- `BinaryTrieNode* child[2]` — Only two children: 0 and 1, corresponding to the bits.
- `int count` — Tracks how many numbers pass through this node. This enables deletion and helps avoid querying deleted numbers.

Insert:

- The number is processed from the most significant bit (MSB) to the least significant bit (LSB). At each bit position, we extract the bit and traverse accordingly. The `count` is incremented at each node along the path.

Query Max XOR:

- At each bit position, we want the opposite bit (1 XOR 0 = 1, 0 XOR 1 = 1). If the opposite child exists, we take it and add `1 << bit` to the result. If not, we take the same bit (adds nothing to the XOR).
- **Why this works:** For each bit, maximizing XOR means choosing the opposite bit. By treating bits from MSB to LSB, we greedily ensure the most significant bits contribute to the XOR first.

Find Max XOR Pair:

- For each number, we first insert it, then query the current Trie for the maximum XOR with that number. The maximum across all numbers is the answer.
- **Alternative:** Insert all numbers first, then query each one. Same result.

10.3 Word Break Using Trie

DP with Trie:

- `dp[i] = true` if `s[0..i-1]` can be segmented into dictionary words.
- For each position `i` where `dp[i]` is true, we start a Trie traversal from `s[i]` onwards. For each position `j` where we encounter a word end, we set `dp[j+1] = true`.
- **Why this is faster than naive DP:** The naive approach checks every substring `s[i..j]` against the dictionary, which is $O(N^2)$ hash lookups. The Trie approach breaks early when a prefix doesn't match any dictionary word, pruning many invalid substrings.

10.4 Aho-Corasick

Building the Automaton:

1. **Trie construction:** Insert all patterns into a standard Trie.
2. **Failure links (BFS):** For each node, compute the longest proper suffix that is also a prefix in the Trie. This is done level by level using a queue.
 - For root's children, failure link = root.
 - For deeper nodes: `fail[node] = go(fail[parent], char)`, where `go` follows the character from the failure node if it exists, otherwise follows failure links recursively.
3. **Output links:** Each node's output is the union of its own patterns (if it's a word end) and the output of its failure node. This ensures we don't miss patterns that end at a suffix position.

Searching:

- Traverse the text character by character. At each step, follow the child edge if it exists; otherwise, follow failure links until a matching edge is found.
- At each position, report all patterns in the current node's output list.
- **Why this is $O(N + M)$:** Each character in the text advances the automaton. Failure links are followed at most once per character (amortized). The total work is proportional to text length + total pattern length.

10.5 Persistent Trie

Core Idea:

- Each insertion creates a new root node. The new root and all nodes along the insertion path are newly created. All other nodes are shared with the previous version.
- **Why this is space-efficient:** Only $O(L)$ new nodes are created per insertion, where L is the word length. The total space is $O(\text{total unique nodes across all versions})$, which is bounded by $O(\text{total words} \times \text{average word length})$.

Key operations:

- When creating a new node, copy all child pointers from the corresponding node in the previous version. Then update only the child pointer that the insertion path follows.
- The `count` field tracks how many words pass through this node in the current version, enabling prefix count queries on any version.

11. Complexity Analysis

11.1 Basic Trie

Operation	Time Complexity	Space Complexity
Insert	$O(L)$	$O(L \times \text{alphabet_size})$
Search	$O(L)$	$O(1)$
StartsWith	$O(L)$	$O(1)$
Delete	$O(L)$	$O(1)$

Operation	Time Complexity	Space Complexity
Autocomplete	$O(L + K)$	$O(K)$
Overall	$O(N \times L)$ build	$O(N \times L \times \text{alphabet_size})$

Where:

- L = length of word / prefix
- N = number of words
- K = number of autocomplete suggestions
- alphabet_size = 26 for lowercase letters

11.2 XOR Trie

Operation	Time Complexity	Space Complexity
Insert	$O(B)$	$O(B)$
Query Max XOR	$O(B)$	$O(1)$
Find Max XOR Pair	$O(N \times B)$	$O(N \times B)$
Delete	$O(B)$	$O(1)$

Where:

- B = number of bits (32 for 32-bit integers, 31 for non-negative)
- N = number of elements

11.3 Word Break Using Trie

Operation	Time Complexity	Space Complexity
Build Trie	$O(M)$	$O(M)$
Word Break check	$O(N^2)$ worst, $O(N \times L_{\text{avg}})$ average	$O(N)$

Where:

- M = total length of all dictionary words
- N = length of string
- L_{avg} = average word length in dictionary

11.4 Aho-Corasick

Operation	Time Complexity	Space Complexity
Build Trie	$O(M)$	$O(M \times \text{alphabet_size})$
Build Failure Links	$O(M \times \text{alphabet_size})$	$O(M)$
Search	$O(N + M + K)$	$O(1)$

Where:

- M = total length of all patterns
- N = length of text
- K = number of matches reported
- `alphabet_size` = 26 for lowercase letters

11.5 Persistent Trie

Operation	Time Complexity	Space Complexity
Insert (new version)	$O(L)$	$O(L)$ per version
Search in version	$O(L)$	$O(1)$
Prefix count in version	$O(L)$	$O(1)$
Total	$O(V \times L)$	$O(V \times L)$

Where:

- V = number of versions
- L = word length

12. Common Patterns

Pattern 1: Dictionary / Word Search

How to identify: Problem involves adding words and searching for them efficiently, often with a prefix constraint.

General approach: Implement a standard Trie with insert, search, and startsWith.

Example problems:

- Implement Trie (LeetCode 208)
- Design Add and Search Words Data Structure (LeetCode 211) — wildcard dot `.` matching
- Word Dictionary (GFG)

Pattern 2: Autocomplete / Search Suggestions

How to identify: Given a prefix, find all words that start with that prefix. Often involves returning top-k suggestions.

General approach:

1. Navigate to the prefix node.
2. DFS/BFS to collect all words.
3. If top-k needed, use a priority queue or sort by frequency.

Example problems:

- Search Suggestions System (LeetCode 1268)
- Implement Magic Dictionary (LeetCode 676)
- Autocomplete System (LeetCode 642)

Pattern 3: Maximum XOR with Binary Trie

How to identify: Given an array, find two elements with maximum XOR. Or query max XOR with a given number.

General approach:

1. Insert all numbers into a binary Trie (MSB to LSB).
2. For each number, greedily choose the opposite bit at each position.
3. Track the maximum XOR found.

Example problems:

- Maximum XOR of Two Numbers in an Array (LeetCode 421)
- Maximum XOR With an Element From Array (LeetCode 1707)
- Maximum XOR Subarray (GFG / Codeforces)

Pattern 4: Word Break / String Segmentation

How to identify: Given a string and a dictionary, determine if the string can be segmented into dictionary words. Or find all possible segmentations.

General approach:

1. Insert dictionary words into a Trie.
2. Use DP where $dp[i]$ represents whether $s[0..i-1]$ is breakable.
3. For each $dp[i] = \text{true}$, traverse the Trie from i onwards to mark $dp[j] = \text{true}$.

Example problems:

- Word Break (LeetCode 139)
- Word Break II (LeetCode 140) — find all segmentations
- Concatenated Words (LeetCode 472)

Pattern 5: Multi-Pattern Matching (Aho-Corasick)

How to identify: Need to find all occurrences of multiple patterns in a single text in one pass.

General approach:

1. Build Trie from all patterns.
2. Compute failure links (BFS).
3. Traverse text once, reporting matches at each position.

Example problems:

- Aho-Corasick (standard implementation)
- Multiple Pattern Matching (SPOJ / Codeforces)

- Find all occurrences of patterns in a string (various)

Pattern 6: Longest Common Prefix

How to identify: Find the longest string that is a prefix of all given strings.

General approach:

1. Insert all strings into a Trie.
2. The longest common prefix is the path from root to the first node with more than one child (or leaf).

Example problems:

- Longest Common Prefix (LeetCode 14)
- Longest Common Prefix in an array of strings (GFG)

Pattern 7: Prefix Count / Frequency

How to identify: Count how many words have a given prefix. Or count how many times a word was inserted.

General approach:

- Maintain a **count** field in each node.
- Increment count during insertion.
- Query count by traversing to the prefix node.

Example problems:

- Prefix match count (GFG)
- Map Sum Pairs (LeetCode 677)
- Count of words with given prefix (various)

Pattern 8: XOR Subarray / Range Queries

How to identify: Find subarray with maximum XOR, or query max XOR within a specific range.

General approach:

- Compute prefix XOR array.
- Use binary Trie to find max XOR with prefix[i] among prefix[0..i-1].

Example problems:

- Maximum XOR Subarray (Codeforces / GFG)
- Maximum XOR of Two Numbers in a Range (LeetCode 1707)

Pattern 9: Persistent Trie for Range Queries

How to identify: Need to query past states of the Trie, or need to answer queries on subarrays.

General approach:

- Build persistent versions of the Trie.

- Answer queries on version V (representing state after processing first V elements).

Example problems:

- Range Xor Queries (advanced)
 - Persistent Trie for maximum XOR in subarray (Codeforces)
-

13. Common Mistakes

13.1 Forgetting the `isEnd` Flag

Mistake: Searching for a word and returning `true` just because the path exists, even if the word was never actually inserted.

Example: Trie contains "catastrophe". Searching for "cat" returns `true` because the path $c \rightarrow a \rightarrow t$ exists.

Fix: Always check `isEnd` in search. `startsWith` is the only operation that doesn't need it.

13.2 Not Handling `isEnd` During Deletion

Mistake: Deleting a node's `isEnd` flag without checking if other words share the same path.

Example: Deleting "cat" from a Trie with {"cat", "catastrophe"}. Removing the 't' node completely would break "catastrophe".

Fix: Only delete nodes that are not shared (no other children and not the end of another word).

13.3 Wrong Alphabet Size

Mistake: Using wrong array size for children. For lowercase letters, it's 26. For uppercase, 26. For digits, 10. For all ASCII, 256.

Fix: Use the correct alphabet size. For flexible size, use a hash map (`unordered_map<char, TrieNode*>`).

13.4 Memory Leak in C++

Mistake: Not deleting Trie nodes, causing memory leaks, especially in competitive programming environments with large inputs.

Fix: Implement a destructor that recursively deletes all children. Or use smart pointers.

13.5 Incorrect Bit Order in XOR Trie

Mistake: Processing bits from LSB to MSB instead of MSB to LSB.

Why it matters: The greedy approach works for XOR because higher bits contribute more to the result. Processing from MSB ensures we make the most important decisions first.

Fix: Always process from MSB (bit 31 or 30) down to 0.

13.6 Not Handling Negative Numbers in XOR Trie

Mistake: Assuming numbers are non-negative. For signed integers, the sign bit (bit 31) matters.

Fix: Use 31 bits for non-negative, 32 bits for signed. Or use `unsigned int` to avoid sign issues.

13.7 Aho-Corasick: Missing Failure Link Merging

Mistake: Building failure links but not merging output from failure nodes.

Example: When matching "he" and "she", if the automaton is at a node that represents "she", it should also report "he" (since "he" is a suffix of "she").

Fix: After setting failure links, merge the output of the failure node into the current node's output.

13.8 Aho-Corasick: Not Handling Missing Edges

Mistake: During search, when a character doesn't exist at the current node, restarting from root instead of following failure links.

Fix: Missing edges should point to the failure link's child for that character (or root if none exists). This is the "goto" function optimization.

13.9 Word Break: Missing Early Termination

Mistake: In the DP + Trie approach, not breaking out of the inner loop when a prefix doesn't match any dictionary word.

Fix: If the current character doesn't exist in the Trie, break — no longer prefix can match.

13.10 Persistent Trie: Not Copying Sibling Pointers

Mistake: When creating a new node along the insertion path, only copying the child pointer that the path follows, not the sibling pointers.

Fix: When creating a new version of a node, copy all its child pointers from the previous version first, then update the one on the insertion path.

14. Edge Cases

Basic Trie

Edge Case	Example	Expected Behavior
Empty string	<code>""</code>	Insert: root becomes <code>isEnd = true</code> . Search: returns <code>true</code> if inserted.
Single character	<code>"a"</code>	Works normally. Only one level deep.
Prefix of another word	<code>"cat"</code> in <code>{"cat", "catastrophe"}</code>	Both words exist. The 't' node is <code>isEnd = true</code> and also has a child 'a'.
Duplicate inserts	Insert <code>"cat"</code> twice	Second insert is harmless. <code>isEnd</code> stays true.

Edge Case	Example	Expected Behavior
Word not found	Search "dog" in empty Trie	Returns false.
Case sensitivity	"Cat" vs "cat"	Different words if case matters. Normalize to lowercase.
Very long word	10 ⁵ characters	Works but depth is large. Recursive approaches may stack overflow.
Unicode characters	Beyond 'a'-'z'	Use hash map instead of array.

XOR Trie

Edge Case	Example	Expected Behavior
Single element	[5]	Max XOR = 0 (5 XOR 5)
All zeros	[0, 0, 0]	Max XOR = 0
All same	[7, 7, 7]	Max XOR = 0
Two elements	[1, 2]	Max XOR = 3
Large numbers	Up to 10 ⁹	Use 31 bits (or 30 for 2 ³¹ safe)
Negative numbers	[-5, 3]	Use 32 bits, handle sign bit
Zero	[0, 5]	Works normally. 0 XOR 5 = 5.

Word Break

Edge Case	Example	Expected Behavior
Empty string	"", dict = ["a"]	True (empty string is always breakable)
Single character	"a", dict = ["a"]	True
No match	"abc", dict = ["d", "e"]	False
All characters in dict but not in order	"abc", dict = ["a", "b", "c"]	True (a
Overlapping words	"aaaa", dict = ["a", "aa"]	True
Impossible segmentation	"leetcode", dict = ["lee", "code"]	False ("lee" + "code" leaves "t" unmatched)

Aho-Corasick

Edge Case	Example	Expected Behavior
Empty patterns	[]	No matches found

Edge Case	Example	Expected Behavior
Empty text	<code>""</code>	No matches found
Single pattern	<code>["abc"], text = "abcabc"</code>	Two matches at positions 0 and 3
Overlapping patterns	<code>["aa", "aaa"], text = "aaaa"</code>	"aa" at 0,1,2; "aaa" at 0,1
Nested patterns	<code>["a", "ab", "abc"], text = "abc"</code>	"a" at 0, "ab" at 0, "abc" at 0
Pattern at text boundary	<code>["ab"], text = "ab"</code>	Match at position 0
Identical patterns	<code>["a", "a"]</code>	Two matches at same position (both indices reported)

Persistent Trie

Edge Case	Expected Behavior
Query version 0 (empty)	No words, search returns false
Query version > current	Return false or handle gracefully
Insert same word in multiple versions	Each version has its own state
Large number of versions	Space grows linearly with versions

15. Variations

15.1 Compressed Trie / Radix Tree (PATRICIA Trie)

What changes: Instead of each edge representing a single character, edges can represent a sequence of characters (a string). Nodes with only one child are merged with their parent.

When it is used: When memory is critical and strings have long common prefixes. More space-efficient than a standard Trie.

Importance: Medium. Useful for IP routing (longest prefix matching). Less common in CP but appears in system design.

15.2 Suffix Trie

What changes: A Trie built from all suffixes of a single string. Allows substring search in $O(L)$ time.

When it is used: For substring queries, pattern matching, finding longest repeated substring, longest common substring.

Importance: High for CP. Often replaced by suffix array + LCP array for better memory efficiency.

15.3 Ternary Search Tree (TST)

What changes: Each node has three children: left (smaller), middle (equal), right (larger). Like a BST of characters instead of an array of 26.

When it is used: When memory is at a premium and the alphabet is large. TSTs use $O(3 \times \text{nodes})$ space instead of $O(26 \times \text{nodes})$.

Importance: Medium. Good for dictionary implementations with large alphabets.

15.4 XOR Trie (Binary Trie)

What changes: Stores binary representation of numbers. Each node has at most 2 children (0 and 1).

When it is used: For maximum XOR pair/subarray problems, XOR queries, bit manipulation.

Importance: Very high. Essential for CP. Frequently appears in Codeforces and LeetCode hard problems.

15.5 Persistent Trie

What changes: Multiple versions of the Trie are maintained. Each insertion creates a new root, sharing unchanged subtrees.

When it is used: For problems involving range queries, time travel, or versions of a dictionary. Especially useful for range XOR queries.

Importance: High for CP. Advanced technique for hard problems.

15.6 Aho-Corasick Automaton

What changes: A Trie augmented with failure links to create a finite automaton for multi-pattern matching.

When it is used: For finding all occurrences of multiple patterns in a text in a single pass.

Importance: Very high. Essential for CP string problems. Appears in Codeforces, AtCoder hard problems.

15.7 Trie with Frequency / Count

What changes: Each node stores a **count** of how many words pass through it. Supports prefix count queries.

When it is used: For problems like "count words with a given prefix", "map sum pairs", or frequency-based autocomplete.

Importance: High. Simple extension that unlocks many problems.

15.8 Trie with Wildcard Support

What changes: Search supports **.** (dot) as a wildcard matching any character. Uses DFS/backtracking for wildcards.

When it is used: For problems like LeetCode 211 (Design Add and Search Words Data Structure) where patterns contain dots.

Importance: High. Common interview question.

16. Related Algorithms / Data Structures

Trie vs Hash Map

Aspect	Trie	Hash Map
Prefix search	$O(L)$	Not supported
Exact match	$O(L)$	$O(1)$ average
Space	$O(N \times L \times \text{alphabet})$	$O(N \times L)$
Ordered traversal	Yes (lexicographic)	No
Memory per character	26+ pointers	String stored once
Best for	Prefix queries, autocomplete	Simple lookups

Choose Trie when: You need prefix queries, autocomplete, or lexicographic ordering. **Choose Hash Map when:** You only need exact match lookups and memory/speed is critical.

Trie vs Suffix Tree

Aspect	Trie	Suffix Tree
Built from	Set of strings	All suffixes of one string
Substring search	$O(L \times \text{alphabet})$	$O(L)$
Memory	Moderate	High (20-30 bytes per char)
Construction	$O(N \times L)$	$O(N)$ (Ukkonen)
Best for	Dictionary, prefix queries	Substring queries on one string

Choose Trie for: Dictionary of multiple strings, prefix queries. **Choose Suffix Tree for:** Substring queries on a single long string.

Trie vs Balanced BST

Aspect	Trie	Balanced BST
Prefix search	$O(L)$	$O(L \times \log N)$
Exact match	$O(L)$	$O(L \times \log N)$
Memory	$O(N \times L \times \text{alphabet})$	$O(N \times L)$
Ordering	Built-in lexicographic	Requires custom comparator

Choose Trie for: String-specific operations (prefix, lexicographic). **Choose BST for:** General-purpose ordered set/map.

Aho-Corasick vs KMP

Aspect	Aho-Corasick	KMP
Patterns	Multiple	Single
Time	$O(N + M + K)$	$O(N + M)$
Space	$O(M \times \text{alphabet})$	$O(M)$
Preprocessing	BFS for failure links	Failure function

Choose Aho-Corasick for: Multiple pattern matching. **Choose KMP for:** Single pattern matching.

Trie + DP (Word Break) vs HashSet + DP

Aspect	Trie + DP	HashSet + DP
Time	$O(N^2)$ worst, better with pruning	$O(N^2)$
Early termination	Yes (prefix check)	No
Memory	$O(M)$ for Trie	$O(M)$ for set
Implementation	More complex	Simple

Choose Trie + DP for: Large dictionaries where many prefixes don't exist. **Choose HashSet + DP for:** Small dictionaries or when simplicity matters.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement Trie (Prefix Tree)	LeetCode 208	Basic Trie with insert, search, startsWith	Easy
Longest Common Prefix	LeetCode 14	Find longest common prefix using Trie or sorting	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Maximum XOR of Two Numbers in an Array	LeetCode 421	Binary Trie for max XOR pair	Medium
Word Break	LeetCode 139	Trie + DP for string segmentation	Medium
Search Suggestions System	LeetCode 1268	Trie + DFS for autocomplete with top-3 results	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Word Break II	LeetCode 140	Trie + DP + backtracking for all segmentations	Hard
Maximum XOR With an Element From Array	LeetCode 1707	Binary Trie with offline queries + sorting	Hard
Design Add and Search Words Data Structure	LeetCode 211	Trie with wildcard <code>.</code> matching using DFS	Medium-Hard
Concatenated Words	LeetCode 472	Trie + DP for multi-word concatenation	Hard

Codeforces / CP Practice

Problem	Platform	Main Idea	Difficulty
XOR Tree	Codeforces	Binary Trie + DP on XOR	1900
Aho-Corasick Automaton	SPOJ	Multi-pattern matching	Medium
Maximum XOR Subarray	CodeChef	Binary Trie + prefix XOR	Medium
Persistent Trie	CodeChef	Persistent Trie for range queries	Hard
String Set Queries	Codeforces	Aho-Corasick with delete support	2400

18. Interview Explanation

Here's a concise explanation you can deliver in an interview:

"A Trie, or prefix tree, is a tree data structure used to store strings efficiently, especially when we need to support prefix-based queries."

The core idea is simple: instead of storing whole strings, we store them character by character along tree paths. The root represents an empty string, and each edge represents a character. A node with the **isEnd** flag marks the end of a complete word. Common prefixes are shared, so inserting 'cat', 'car', and 'cab' only creates one 'c' node and one 'a' node.

The main operations are:

- **Insert** — $O(L)$ time, where L is the word length. We walk through each character, creating nodes if they don't exist, and mark the final node as a word end.
- **Search** — $O(L)$ time. We follow the same path and check the **isEnd** flag.
- **StartsWith** — $O(L)$ time. We just check if the path exists, without needing **isEnd**.

The key advantage over a hash map is that prefix queries are $O(L)$ instead of impossible. The trade-off is memory — each node uses an array of 26 pointers, which can be wasteful for sparse data.

For more advanced problems, we can extend the Trie:

- **A Binary Trie** stores numbers as 32-bit strings (each node has only 0 and 1 children). This lets us solve maximum XOR pair problems in $O(N * 32)$ instead of $O(N^2)$.
- **Aho-Corasick** adds failure links to the Trie, turning it into a finite automaton that can find all occurrences of multiple patterns in a single pass through the text.
- **A Persistent Trie** creates a new version on each insertion, sharing unchanged subtrees. This allows querying any historical state.

In practice, I'd use a Trie when the problem involves prefix matching, autocomplete, or dictionary lookups with prefix constraints. The implementation is straightforward — a node class with an array of children and a boolean flag, and iterative traversal for insert and search."

19. Revision Notes

Basic Trie

- **Key idea:** Tree of characters, common prefixes shared.
- **Node fields:** `children[26]`, `isEnd`.
- **Insert:** Walk characters, create nodes as needed, mark `isEnd`.
- **Search:** Walk characters, return `isEnd` at end.
- **StartsWith:** Walk characters, return `true` if path exists.
- **Complexity:** $O(L)$ per operation, $O(N \times L \times 26)$ space.
- **Common trap:** Forgetting `isEnd` in search.
- **Edge case:** Empty string (root itself is a word end).

XOR Trie (Binary Trie)

- **Key idea:** Store numbers as binary strings (MSB to LSB).
- **Goal:** For each bit, try opposite for max XOR.
- **Complexity:** $O(N \times 32)$ for max XOR pair.
- **Common trap:** Processing bits LSB to MSB (wrong).
- **Edge case:** Single element (XOR = 0).

Word Break

- **Key idea:** DP + Trie. `dp[i]` = breakable up to position `i`.
- **For each `i` where `dp[i]` is true:** traverse Trie from `i` onwards.
- **Early termination:** break if prefix doesn't match.
- **Complexity:** $O(N^2)$ worst, but practical pruning.

Aho-Corasick

- **Key idea:** Trie + failure links = automaton.
- **Build:** BFS to compute failure links.
- **Search:** One pass through text, follow failure links on mismatch.
- **Output:** Each node reports patterns ending at its failure chain.
- **Complexity:** $O(N + M + K)$.
- **Common trap:** Not merging output from failure nodes.

Persistent Trie

- **Key idea:** Each insertion creates a new root. Only path nodes are new.
- **Space per version:** $O(L)$ new nodes.
- **Use case:** Range queries, historical state queries.

20. Final Cheat Sheet

Trie — Quick Reference

TRIE CHEAT SHEET																										
WHEN TO USE: • Prefix matching / autocomplete • Dictionary with prefix queries • Multiple strings sharing common prefixes • Binary XOR problems (Binary Trie) • Multi-pattern matching (Aho-Corasick)																										
MAIN OPERATIONS: <table> <tr> <th>Operation</th><th>Complexity</th><th>Key Detail</th></tr> <tr> <td>Insert</td><td>$O(L)$</td><td>Create nodes if missing</td></tr> <tr> <td>Search</td><td>$O(L)$</td><td>Check isEnd at end</td></tr> <tr> <td>StartsWith</td><td>$O(L)$</td><td>Only check path exists</td></tr> <tr> <td>Delete</td><td>$O(L)$</td><td>Recursive, don't remove shared</td></tr> <tr> <td>Autocomplete</td><td>$O(L + K)$</td><td>DFS from prefix node</td></tr> <tr> <td>XOR Max Query</td><td>$O(B)$</td><td>Greedy opposite bit</td></tr> <tr> <td>Aho-Corasick</td><td>$O(N+M+K)$</td><td>Failure links for multi-pat</td></tr> </table>			Operation	Complexity	Key Detail	Insert	$O(L)$	Create nodes if missing	Search	$O(L)$	Check isEnd at end	StartsWith	$O(L)$	Only check path exists	Delete	$O(L)$	Recursive, don't remove shared	Autocomplete	$O(L + K)$	DFS from prefix node	XOR Max Query	$O(B)$	Greedy opposite bit	Aho-Corasick	$O(N+M+K)$	Failure links for multi-pat
Operation	Complexity	Key Detail																								
Insert	$O(L)$	Create nodes if missing																								
Search	$O(L)$	Check isEnd at end																								
StartsWith	$O(L)$	Only check path exists																								
Delete	$O(L)$	Recursive, don't remove shared																								
Autocomplete	$O(L + K)$	DFS from prefix node																								
XOR Max Query	$O(B)$	Greedy opposite bit																								
Aho-Corasick	$O(N+M+K)$	Failure links for multi-pat																								
SPACE: $O(N \times L \times \text{alphabet})$																										
KEY CODE — TRIE NODE: <pre>struct TrieNode { TrieNode* children[26]; bool isEnd; };</pre>																										
KEY CODE — INSERT: <pre>for (char c : word) { int idx = c - 'a'; if (!curr->children[idx]) curr->children[idx] = new TrieNode(); curr = curr->children[idx]; } curr->isEnd = true;</pre>																										

KEY CODE – XOR TRIE QUERY:

```
for (int bit = 31; bit >= 0; bit--) {  
    int b = (num >> bit) & 1;  
    if (curr->child[!b]) { result |= (1<<bit); curr=child[!b]; }  
    else { curr = child[b]; }  
}
```

IMPORTANT EDGE CASES:

- Empty string ("") – root isEnd = true
- Prefix of another word – don't delete shared nodes
- XOR single element – max XOR = 0
- Aho-Corasick overlapping patterns – merge output from fail
- Word Break empty string – dp[0] = true

COMMON TRAPS:

- Forgetting isEnd in search
- Wrong bit order in XOR Trie (MSB → LSB, not LSB → MSB)
- Not deleting Trie nodes in C++ (memory leak)
- Aho-Corasick: not merging failure output
- Word Break: not breaking when prefix doesn't match